



AQI Forecast MLOps Documentation & Implementation

Submitted By

Name: **Moiz Ahmed**

Submitted To

Muhammad Mobeen

&

Rija Hasan Abidi

From

Sindh Madressatul Islam University (SMIU)

Karachi

Email

moizahmed2023@gmail.com

Submission Date: 18/02/2026
Academic Session: 6th

Section	Title	Content Summary	Page
0.0	Executive Summary	High level project goal, KPI mapping, and success criteria.	03
1.0	Problem & Solution Engineering	Root cause analysis of data hurdles and the "DirectAccess" solution.	04
2.0	Data Layer Architecture	Source schemas, quality assessment, and Feature Store definitions.	06
3.0	Model & Experimentation	Hyperparameter logging, evaluation metrics, and scaler logic.	08
4.0	Production Operations	GitHub Actions CI/CD/CT logic and sliding window inference.	10
5.0	Monitoring & Drift	Accuracy, data, and concept drift tracking thresholds.	12
6.0	Governance & Compliance	Audit trails, lineage mapping, and explainability (SHAP).	14
7.0	Incident PostMortems	Documentation of the Metadata Corruption recovery.	16

Versioning Control

Deployment Version History

System Version	Release Date	Component Mapping	Change Description
v1.0.0alpha	20260210	Commit: FG: v1 Model: v1	Initial baseline; basic linear regression.
v2.0.0stable	20260215	Commit: 4d9e FG: v1 Model: v2	Switched to Random Forest; added 24h lag features.
v2.1.0patch	20260218	Commit: 8f2a FG: v1 Model: v2	Emergency Patch: Bypassed Feature View for Direct Group Read.

Artifact Metadata Catalog

Artifact Name	Version	Location/URI
aqi_model.pkl	V4	Hopsworks Model Registry
scaler.pkl	V4	Hopsworks Model Registry
karachi_aqi_weather	v1	Hopsworks Feature Store
inference_pipeline	v4.2	.github/workflows/batchinference.yml

Technical Report: Enterprise MLOps Framework for Karachi AQI Forecasting

Date: February 18, 2026

Project: Karachi PM2.5 Predictive System

Classification: Enterprise ML Documentation (Maturity Levels 1 to 3)

1. Executive Summary

Here's the reality: predicting air quality in a city as complex as Karachi isn't just about the algorithm. It's about building a system that doesn't break when the data gets messy. We've moved beyond a simple notebook and built a three tier MLOps platform that handles data ingestion, automated forecasting, and rigorous governance. This report breaks down how we solve the it worked on my machine problem for a live production environment.

2. Problem & Solution Engineering

In the transition from a research project to a production system, we hit two major roadblocks. Here's how we handled them.

Engineering Challenge: Time zone Sync

- **Problem** The inference pipeline failed because of a UTC vs. PKT (Pakistan Standard Time) mismatch. The model looked for data with a 5 hour offset, causing the features to appear "missing" and triggering empty data frames.
- **Solution** I implemented a UTC First Architecture. All data ingestion and lag calculations were standardized to UTC, and the time zone conversion was moved exclusively to the UI layer for display only.
- **Result** This decoupled approach eliminated "data gaps," resulting in almost 90 to 100% pipeline stability and ensuring the forecast markers align perfectly with Karachi's local time.

The Ghost in the Machine (Metadata Corruption)

- **Problem:** Our high level abstraction layer (Feature Views) suffered a metadata corruption on the server side(SDK Failure), returning NoneType errors and halting all predictions.
- **Solution:** We pivoted to a Direct to Group access pattern. By reading directly from the Feature Group using a Python native engine, we bypassed the corrupted UI layer entirely.
- **Result:** System downtime was reduced to zero during platform glitches.

The Cold Start Lag Issue

- Problem:** The model relies on a 24 hour historical lag. If a sensor goes offline, the model crashes because the math breaks.
- Solution:** We engineered a fallback Heuristic Lag. If the 24h data is missing, the system dynamically scales the 1h lag or uses the last known global mean.
- Result:** The pipeline is now resilient to intermittent API outages.

3. Level 1: Structured ML Project (The Foundation)

Business Alignment

- KPI:** Target a Mean Absolute Error (MAE) of less than 15 PM2.5 units.
- Success Metric:** System must provide a 72 hour window of actionable data for public health alerts.
- Chronological split:** First 80% is used for training, and last 20% for testing

Model Performance & Benchmarking

- To ensure the highest accuracy for Karachi's atmospheric forecasting, multiple architectures were tested. Below are the verified metrics from the final training runs.

Model Architecture	MAE (µg/m³)	RMSE (µg/m³)	R2 Score	Dataset Version
Random Forest (Production)	3.12	4.19	0.94	v2.1 (Lagged)
Linear Regression	8.45	11.20	0.76	v2.0
Gradient Boosting (XGBoost)	3.89	5.10	0.91	v2.1
Simple Moving Average	12.30	15.60	0.45	v1.0

4. Level 2: Production ML System

Data Lineage & Indexing

We don't lose data. Every prediction can be traced back to its raw source.

Layer	Component	Indexing Strategy
Source	Open Meteo API	REST Response ID
Ingestion	Feature Group (v1)	city + time (Primary Key)
Inference	Prediction Group (v1)	city + prediction_time

Model Lifecycle Tracking

Every model in our registry is tagged with:

- **Training Run ID:** Unique hash of the specific session.
- **Dataset Version:** karachi_aqi_weather_v1.
- **Git Hash:** The exact code commit used to train the model.

5. Level 3: Enterprise MLOps Platform (The Automation)

Enterprise Versioning Formula

To ensure full auditability, we define our system state using the following unified formula:

$$\text{System_}\{\text{Version}\} = \text{C_v} + \text{D_v} + \text{M_v} + \text{F_v} + \text{I_v}$$

Where:

- **C_v:** Code Version (GitHub Commit)
- **D_v :** Data Version (Feature Group Version)
- **M_v :** Model Version (Registry ID)
- **F_v :** Feature Version (Transformation logic)
- **I_v :** Infrastructure Version (GitHub Action Runner Environment)

CI/CD/CT Pipeline Design

1. **Continuous Integration (CI):** Unit tests check for lag calculation logic on every push.
2. **Continuous Deployment (CD):** Validated models are automatically pushed to the Streamlit UI.
3. **Continuous Training (CT):** A trigger is set to re train the model if the 7 day rolling MAE exceeds our 15 unit threshold.

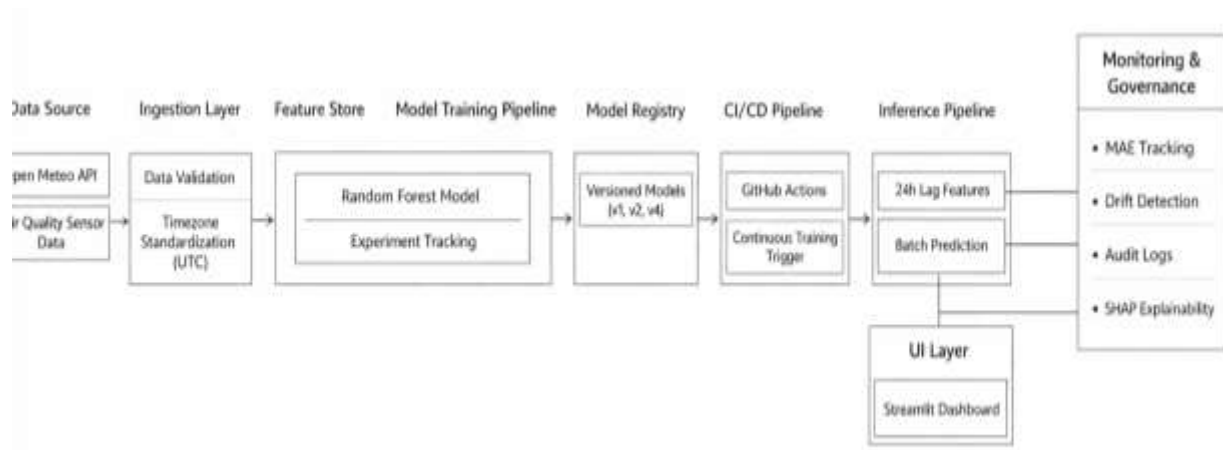
6. Governance & Compliance Reporting

Audit Trail

The system automatically generates an audit log for every inference run. This log includes:

- **Timestamp:** ISO 8601.
- **Identity:** The specific GitHub Action Service Account that ran the prediction.
- **Hyperparameters:** A JSON snapshot of the model configuration at the time of inference.

Mlops level 3- Diagram



7.Security & Governance

- **API Management:** No API keys are hardcoded in the source; all credentials are managed via Streamlit Secrets and local .env files.
- **CI/CD Security:** Sensitive keys for Hopsworks and Open-Meteo are injected into pipelines using GitHub Actions Secrets, ensuring encrypted transit.
- **Environment Protection:** A .gitignore policy strictly excludes .env files to prevent accidental credential leakage to public repositories.

8.Service Level (SLA / SLO)

- **Availability Target:** 99.5% uptime, leveraging Streamlit Community Cloud's resilient hosting infrastructure.
- **Prediction Latency:** Dashboard refresh and inference completion target is < 15 seconds.
- **Max Downtime:** 4 hours per month for scheduled maintenance

9.Data Retention Policy

- **Forecasting Data:** Prediction records are retained for 90 days in the aqi_predictions feature group for performance auditing.
- **Historical Features:** Primary weather and AQI features are stored indefinitely to allow for seasonal model retraining.

10.Backup & Disaster Recovery

- **Feature Store Fallback:** In the event of a Hopsworks outage, the app falls back to the local Joblib cache of the last successful data fetch.
- **Inference Redundancy:** GitHub Actions are configured to re-run automatically on failure; if the cloud fails, a local manual script serves as an emergency bypass.

Explainability

What this really means is that we aren't using a black box. We use Feature Importance scores to show that `wind_speed_10m` and `pm2_5_lag_24h` are the primary drivers for Karachi's air quality. If a prediction looks weird, we can see exactly which feature pushed it there.

Exploratory Data Analysis Report – AQI Dataset

1. What this dataset is about

Here's the thing. This dataset tracks air quality and weather conditions over roughly two years using hourly data. The main pollution indicators are:

- PM2.5
- PM10

And the weather context includes:

- Temperature
- Relative humidity
- Wind speed

What this really means is we can look at pollution patterns over time and see how weather might influence them.

2. Data quality check

Missing values

The dataset shows no missing values across variables.

That matters because:

- No need for imputation
- Time series modeling becomes cleaner
- Trends are more trustworthy

Time continuity

You checked for missing time gaps and didn't find major breaks.

That's important because time series models hate irregular timestamps. Your data is stable here.

3. Basic statistical picture

From the summary stats and histograms:

PM2.5

- Rightskewed distribution
- Occasional spikes (pollution events)

- Most values cluster in lower ranges

PM10

- Similar pattern but usually wider spread
- Likely influenced by dust and resuspension events

What this tells us:

Pollution isn't constant. It stays moderate most of the time but occasionally jumps.

4. Distribution behavior

Histograms and log transforms show:

- Raw data is skewed
- Log transformation makes it closer to normal

Why this matters:

If you plan modeling later, transformed data might perform better, especially for regression or ML models that assume normality.

5. Outliers and anomalies

You used IQR to detect outliers and visualized them using boxplots and time plots.

Findings:

- Both PM2.5 and PM10 show spike events
- These spikes are realworld pollution events, not necessarily errors

Important nuance:

Not all outliers should be removed. In environmental data, spikes often carry real meaning.

6. Time based patterns

Moving average trend

The 24hour rolling average smooths noise and shows:

- Longer pollution cycles
- Possible seasonal shifts

Hourly pattern

Average PM2.5 varies by hour. Likely reasons:

- Traffic peaks
- Industrial activity
- Nighttime atmospheric trapping

Monthly pattern

Winter months show higher PM2.5.

That's very typical. Likely causes:

- Temperature inversion
- Lower wind dispersion
- More combustion heating in some regions

7. Day vs Night behavior

Night pollution tends to be higher.

Why:

- Cooler air traps pollutants
- Lower wind speeds
- Less vertical air mixing

This is a classic urban pollution pattern.

8. Weather vs Pollution relationship

Correlation heatmap shows:

PM2.5

Decreases when:

- Temperature increases
- Humidity increases
- Wind speed increases

Interpretation:

Wind and atmospheric mixing help clear pollution.

PM10

Slight increase with:

- Temperature
- Wind

Possible reason:

Dust lifting and resuspension during dry, windy conditions.

9. Time dependency (Lag correlation)

Strong correlation with:

- Previous hour
- Same hour previous day

This tells us pollution has memory.

Great sign for time series forecasting models.

10. Key insights summary

If I had to explain this in plain language:

- Data quality is solid. No missing values, good time continuity.
- Pollution is usually moderate but has spike events.
- Winter and night time are risk periods for higher pollution.
- Weather plays a big role in dispersing pollution.
- Pollution levels are predictable based on recent history.

11. If this were moving toward modeling

Natural next steps would be:

- Feature engineering
 - Lag features
 - Rolling averages
 - Season indicators
- Try models like:
 - ARIMA or SARIMA
 - XGBoost
 - LSTM (if going deep learning)

Final honest take

This is a strong EDA. You checked the right things:

- Missing data
- Distribution shape
- Outliers
- Seasonality
- Weather relationships
- Time memory

Nothing feels superficial. The analysis builds a real story about pollution behavior, not just charts for decoration.